

系统模块与算法维护手册

System Modules, Algorithms & Change-Control Manual

投影 / UV 合成 / 局部重绘 / 图层 / 保存与版本历史

文档版本	1.2.0
规范状态	当前唯一维护准则 / Normative Baseline
生效日期	2026-08-26
代码盘点基线	ea86557 + 盘点时既有工作区修改
适用对象	两位维护者、Codex、后续开发与测试人员

文档控制

本文档既是系统模块目录，也是算法注册表和强制变更流程。任何行为修改必须先定位 M/UI/ALG ID，再建立 CHG 记录；历史方案不能覆盖本手册的当前真值。

控制项	当前规则
规范范围	模块边界、界面调用、算法参数、图层语义、保存、版本、测试与变更控制
真值原则	代码是实现证据；本手册是维护规范。冲突必须建立变更记录后同时修正。
算法版本	规范版本与运行版本分离；没有持久化标记的算法明确登记为治理缺口。
源码历史	Git + CHANGELOG + docs/changes + ADR；工程 autosave 不能代替源码版本。
默认修改上限	普通 Bug 修复限制在一个大模块、通常不超过 5 个源文件或 300 行净变化。

目录

文档控制	2
目录	3
1. 本文档的地位	7
2. 维护目标与禁止事项	7
2.1 维护目标	7
2.2 明确禁止	7
3. 系统上下文与大模块	8
4. 依赖规则	9
4.1 允许的方向	9
4.2 目标模块目录	10
5. 当前端到端真值	11
5.1 新建与保存	11
5.2 单视图/多视图纹理生成	11
5.3 投影到 UV	11
5.4 局部重绘	11
5.5 内容识别补缝	12
5.6 输出	12
6. 小模块与当前算法清单	13
6.1 M01 工程与工作区	13
6.2 M02 输入与资产	13
6.3 M03 场景、相机与捕获	14
6.4 M04 生成编排	15
6.5 M05 图层领域	16
6.6 M06 实时投影与可见性	17

6.7 M07 UV 烘焙、权重与合并	18
生产算法	18
当前必须保留的已知差异	19
6.8 M08 局部重绘	19
6.9 M09 内容识别补缝	21
6.10 M10 生产流程	21
6.11 M11 输出与集成	22
6.12 M12 状态、历史与版本	22
6.13 M13 平台与性能基础设施	23
7. 版本注册表	24
7.1 算法名称与规范版本	24
7.2 当前版本治理缺口	27
8. 变更分级与 Codex 修改上限	28
9. 标准变更流程	28
9.1 开始前	28
9.2 实施中	28
9.3 完成前	28
10. Git 与提交规范	29
11. 模块验收矩阵	29
12. 当前架构债务与拆分顺序	29
13. 文档维护规则	31
14. 编辑器界面模块总图	32
14.1 截图区域与唯一模块编号	32
14.2 工作区切换的实际含义	34
14.3 面板生命周期和编辑锁	34

15. 每个界面模块的操作调用表	35
15.1 UI-01/UI-02/UI-03 工程头、流程和设置	35
15.2 UI-04 对象面板	35
15.3 UI-05 生成面板	36
15.4 UI-06/UI-07/UI-08 视口	36
15.5 UI-09 图层面板顶部动作	36
15.6 UI-09 图层行操作	37
15.7 UI-10 局部三步工具条	37
16. 图层类型、角色和效果唯一判定	38
16.1 四个维度不能混用	38
16.2 截图中各类图层的准确差别	38
16.3 当前层序合成规则	39
17. 投影算法审计 (关键模块)	40
17.1 调用图	40
17.2 算法登记卡	40
17.3 生产参数基线	41
17.4 Top-3 权重精确定义	41
17.5 GPU 资源预算	41
18. UV 合成与合并算法审计 (关键模块)	43
18.1 右侧剪刀的完整事务	43
18.2 算法登记卡	43
18.3 GPU 与 CPU 回退不是同一参数副本	44
18.4 UV 输出边界	44
19. 局部重绘算法审计 (关键模块)	45
19.1 主路径状态机	45
19.2 算法登记卡	45

19.3 远端没有收到什么	46
19.4 Seam Harmonization v2 参数	46
19.5 截图中 “一行重绘” 的双层实现	47
20. Ctrl+S 保存实现审计	48
20.1 一句话答案	48
20.2 精确调用链	48
20.3 快照包含什么	48
20.4 工程目录和文件格式	49
20.5 保存保护与用户可见语义	49
21. 算法调用审计的维护格式	50
22. 修订历史	51

1. 本文档的地位

本文档是 LI3D 的模块边界、当前算法、代码修改流程、版本规则和验收要求的唯一规范性入口。

- 修改代码前必须先按模块 ID 在本文档中定位责任边界。
- 本文档描述“当前实现”时，以盘点日实际代码为依据；描述“目标边界”时，不代表代码已经完成迁移。
- 其他功能文档、日期总结、性能 Phase 文档和合并记录都是补充资料，不得覆盖本文档的模块边界和变更规则。
- 如果代码、本文档和历史文档互相冲突，禁止静默选择其中一个继续修改。先建立变更记录，写清冲突、实际运行证据和处理决定，再同时校正代码与本文档。
- 每次算法语义、数据契约、模块边界或默认参数发生变化，必须在同一个变更中更新本文档的相应章节。

本文档不要求每个小 Bug 都进行大规模重构。它要求每次改动都能回答：改的是哪个模块、为什么改、影响哪些契约、如何验证、如何回退、历史记录在哪里。

2. 维护目标与禁止事项

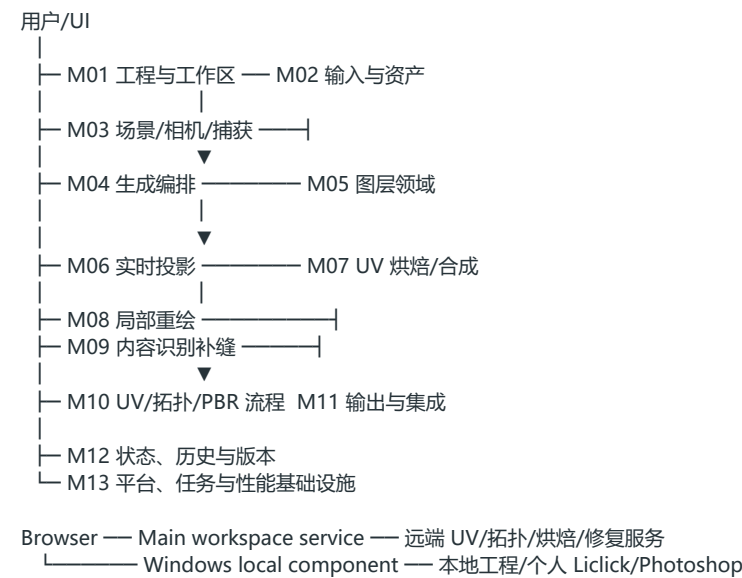
2.1 维护目标

1. 一个问题只进入一个明确的主模块，由其他模块通过公开契约配合。
2. 算法、UI、状态、持久化和外部服务分层，不能继续堆入同一个超大组件。
3. 每次行为变化都有可追踪的变更单、提交、测试结果和版本号。
4. 同一算法的实时预览、GPU 烘焙和 CPU 参考实现具有明确的共同语义与校准测试。
5. 两个人可以并行工作，但不能在同一问题上直接覆盖对方的工作区或主分支。

2.2 明确禁止

- 禁止为了修复一个 Bug 顺手重命名、搬迁或格式化不相关代码。
- 禁止未经说明跨越两个以上大模块修改内部实现。
- 禁止把 3D、图像处理、UV 或权重算法继续写进 React 页面和面板组件。
- 禁止 UI 直接拼接服务端协议，或算法直接操作 Toast、对话框、路由和翻译文案。
- 禁止用降低分辨率、关闭质量门、跳过深度/法线检查来伪装性能修复，除非它是用户明确选择的模式。
- 禁止修改旧的日期历史文档来“证明”新行为一直存在。历史事实只追加，不回写。
- 禁止两个人长期直接在 `master` 上开发；一个问题对应一个短生命周期分支和一组聚焦提交。
- 禁止让 Codex 在未说明影响范围时进行目录级重构、大文件拆分或数据迁移。

3. 系统上下文与大模块

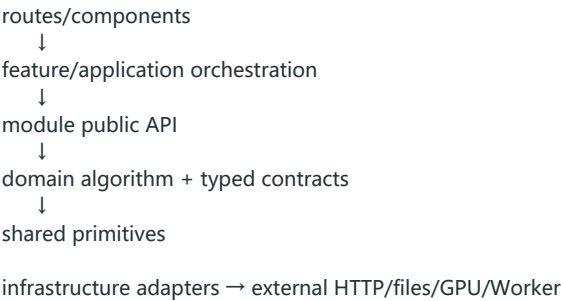


M01 工程与工作区
责任: 新建、加载、保存、复制、删除、资产清单、工程冲突保护
不负责: 纹理算法和 3D 渲染
当前主要位置: <code>services/workspaceApiClient.ts</code> 、 <code>projectFileService.ts</code>
M02 输入与资产
责任: 模型、参考图、已有工程与二进制资产的接收和预处理
不负责: 生成、投影、UV 合成
当前主要位置: <code>engine/loaders</code> 、参考图面板与预处理服务
M03 场景/相机/捕获
责任: 模型归一化、变换、相机、Color/Mask/Depth/Normal 捕获
不负责: 远端生成和图层持久化
当前主要位置: <code>engine/scene</code> 、 <code>engine/capture</code> 、 <code>engine/viewport</code>
M04 生成编排
责任: 单视图、多视图、参考图补全、任务提交/恢复/取消
不负责: 投影采样和 UV 像素合成
当前主要位置: <code>GeneratePanel.tsx</code> 、生成 API clients/services
M05 图层领域
责任: 新增图层、类型/角色、顺序、显隐、透明度、混合、调整
不负责: 生成任务和模型导出
当前主要位置: <code>types/layer.ts</code> 、 <code>stores/layerStore.ts</code> 、 <code>engine/layers</code>
M06 实时投影
责任: 相机投影、表面可见性、实时多视图权重、GPU 预览材质
不负责: 最终工程保存和远端生成
当前主要位置: <code>engine/projection</code>

M07 UV 烘焙与合成
责任: 投影到 UV、质量加权、Overlay、拓扑约束补洞、接缝、合并
不负责: UV 展开、远端 PBR 通道烘焙
当前主要位置: engine/bake、engine/layers
M08 局部重绘
责任: 画笔蒙版、局部生成、表面锁定、边缘融合、重绘提交
不负责: 普通自由绘画、内容识别全局补缝
当前主要位置: engine/localRepaint、ViewportCanvas.tsx、EditorPage.tsx
M09 内容识别补缝
责任: 识别 UV 低覆盖区域、构建表面拓扑、邻近传播、稀疏修补层
不负责: 调用生成模型重新画整张图
当前主要位置: engine/contentAware
M10 生产流程
责任: Texture → Retopology → UV → Bake 追加式交接、远端任务
不负责: 浏览器内投影算法
当前主要位置: workflow/bake features、server proxies
M11 输出与集成
责任: BaseColor/模型/快照/视频/包导出、Photoshop 桥接
不负责: 编辑器状态决策
当前主要位置: engine/export、integrations/photoshop-uxp
M12 状态、历史与版本
责任: Zustand 状态、撤销重做、工程版本、流水线 revision、源码变更记录
不负责: 图像计算本身
当前主要位置: stores、types/project.ts、projectPipeline.ts
M13 平台与性能
责任: 身份、传输选择、任务历史、Worker、缓存、调度、遥测
不负责: 决定算法产品语义
当前主要位置: web/server services、engine/performance、workers

4. 依赖规则

4.1 允许的方向



- 1. 页面和组件只能编排用户交互，算法必须下沉到所属模块。
- 2. 模块间通过类型化公开 API 连接；迁移完成后禁止从另一模块深层导入内部文件。
- 3. 纯算法不得导入 React、Zustand store、Toast、i18n、路由或 HTTP client。
- 4. Store 只保存状态和执行小型原子更新，不承载几百行图像算法。
- 5. 外部服务只能通过 adapter/client 接入；业务编排不得依赖某个服务的原始 JSON 结构。
- 6. Worker 与主线程必须共享显式 request/response 类型，并保持同一算法版本。
- 7. GPU 路径是生产实现时，CPU 路径作为可比较的参考实现；两者差异必须是已记录的决定，不能靠复制常量长期漂移。

4.2 目标模块目录

现有代码不在本次治理工作中整体搬迁。后续按实际修改逐步收敛到以下结构：

```
apps/web/src/modules/<module-name>/
index.ts      # 唯一公开入口
contracts.ts  # 输入、输出、错误和版本
domain/       # 纯算法，不依赖 UI/store/network
application/  # 用例编排
infrastructure/ # GPU、Worker、HTTP、文件适配器
ui/           # 本模块面板和 hooks
__tests__/_/
README.md     # 仅记录该模块实现细节，规范仍以本文档为准
```

迁移顺序按风险而不是按目录字母排序：先提取纯函数和契约，再提取编排，最后缩小超大 UI/渲染组件。不得一次性全仓搬家。

5. 当前端到端真值

5.1 新建与保存

1. Web 调用 `createProject`。
2. 工作区服务按用户创建工程目录、`project.liclick.json`、资产子目录、导出、缩略图和 `autosave` 目录。
3. 新工程默认包含空对象/参考/捕获/生成/图层/烘焙数组，默认 2K、PBR、透视、sRGB，`workspaceVersion=0.6.0`。
4. 前端修改先进入 Zustand 与 `Project` 内存对象并标记 `dirty`；二进制资产先写入分类目录，再把工程内 URL 归一化为相对路径。
5. 服务端按 `userId:projectId` 串行保存，拒绝旧 `updatedAt` 覆盖新快照、意外空场景覆盖和意外清空全部图层。
6. JSON 使用临时文件加 `rename` 原子替换；Windows 短时锁会按 25/50/100/200/400/800ms 重试。
7. 成功保存后写一份 `autosave`，最多保留 5 份。

保存不是源码版本管理。工程 `autosave` 只保护用户工程；算法源码历史必须走 Git 和本规范的变更记录。

5.2 单视图/多视图纹理生成

1. 用户选择单视图或多视图材质参考。只有单图时，系统先生成一张成对的多视图参考并写回 `reference group`。
2. 单视图使用当前相机；多视图使用预设或自定义相机列表。每个视角都独立捕获并独立提交 `single-view` 生成任务。
3. 整批 `Capture` 必须在远端任务开始前一次性持久化，避免完成结果找不到原相机。
4. 每个任务保存 `textureBatchId`、`captureId`、对象 ID、捕获时对象矩阵、材质参考、相机视角和任务 ID。
5. 任务可轮询、取消、恢复。各视角完成后按队列持久化，并自动创建 `Projected Layer`。
6. 多视图至少成功一个投影后，自动触发一次内容识别补缝；补缝失败不删除已完成投影。

当前“单视图”和“多视图”的差别是相机数量及批次编排，不是两套远端生成协议。

5.3 投影到 UV

1. 使用图层保存的捕获相机和对象矩阵恢复生成时几何状态。
2. UV 三角形在 UV 空间栅格化，并重建每个 `texel` 的世界位置与法线。
3. 世界点经 `Projection * View` 进入生成相机裁剪空间。
4. 依次执行视锥、源 alpha、蒙版、线性视深、捕获法线、面向、图层透明度/强度和图像边缘门控。
5. 普通投影进入 Top-3 质量合成；局部重绘和 `Overlay` 在普通结果后按专用规则叠加。
6. 根据调用方决定是否执行拓扑约束 `gutter`、内部缺口、接缝和锐化。
7. 返回 `straight RGBA`、覆盖率、报告和可选 PNG；调用方决定新建 UV 层、写入空白 UV 层、缓存、导出或仅分析。

5.4 局部重绘

当前有两条仍在代码中的路径，修改时必须先确认目标路径：

- 主路径：生成面板的 `ModelView` 局部生图。冻结相机与对象矩阵，捕获 1K 白模、1K Flat 当前贴图和多视图材质参考；画笔蒙版只负责本地回贴，不送给该生成服务；深度保护与远端生成并行获取。结果以保存的画笔蒙版、深度和表面锁定规则投回模型。
- 兼容路径：`LocalRepaintDialog` 的图像编辑 `provider` 工作流。它构造 `edit/protect/hole mask`、ROI、远端编辑任务和合并结果，可更新原投影图或烘焙独立 UV 修补层。

二者不能在一次“局部重绘修复”中被当成同一实现一起大改。变更单必须写 M08-A 或 M08-B。

5.5 内容识别补缝

1. 将普通可见投影按生产质量规则烘焙为带覆盖置信度 alpha 的临时 UV 图。
2. 把已有内容修补层作为稀疏 underlay 合成，不能覆盖已投影像素。
3. 从模型 UV 生成 conservative topology mask、strict core mask、region、conflict 和物理 seam links。
4. alpha 小于硬阈值的 strict-core texel 作为缺口；允许增长一圈 weak texel，过滤孤立噪点但保留长窄接缝。
5. Worker 从合格边界颜色源进行 $O(N + \text{seam links})$ 最近源传播，只能在同 region 内移动，最多跨一个验证物理接缝。
6. 不可达区域保持透明；禁止全局平均色填充。
7. 输出仅包含修复像素的稀疏 UV underlay，并在纹理解码/GPU 预热完成后原子发布。

5.6 输出

导出不直接抓取当前屏幕作为模型贴图。它先解析当前对象的可见 UV 层和投影栈，优先复用签名匹配的 bake cache；缓存不匹配时按需生成精确 BaseColor，再交给 GLB/FBX/OBJ 等 exporter。输出方向保持 `texture.flipY=false` 的项目约定。

6. 小模块与当前算法清单

下面每个 ID 都是变更记录、提交 scope 和评审时使用的稳定标识。

6.1 M01 工程与工作区

M01-01 新建工程

当前实现: 创建用户隔离目录与默认 Project, 初始化资产清单

输入 → 输出: 名称/文件夹 → 工程 JSON

主要代码: workspaceApiClient.ts、projectFileService.ts

最低验证: server build + 新建/重开 smoke

M01-02 加载工程

当前实现: 读取 JSON、修补缺失 layer asset 引用、把相对路径解析为可访问 URL

输入 → 输出: projectId → 完整工程

主要代码: projectFileService.ts

最低验证: editor route/load tests

M01-03 保存工程

当前实现: 串行、冲突保护、显式删除、相对路径归一化、原子替换

输入 → 输出: 内存 Project → project.liclick.json

主要代码: projectFileService.ts、atomicFileService.ts

最低验证: pipeline persistence + save conflict cases

M01-04 自动备份

当前实现: 每次成功保存追加一份时间戳快照, 只留最新 5 份

输入 → 输出: Project → autosave 文件

主要代码: autosaveService.ts

最低验证: 人工检查轮转

M01-05 工程资产

当前实现: models/references/captures/generations/layers/baked 分类写入, 禁止 blob/绝对路径落盘

输入 → 输出: Blob/Data URL/远端 URL → 相对资产

主要代码: workspace/asset services

最低验证: local server smoke

M01-06 删除/复制/移动

当前实现: 删除工程进入用户 trash; 复制生成新 ID/slug; 移动只改 folder

输入 → 输出: projectId → 新工程或位置

主要代码: projectFileService.ts

最低验证: projects API smoke

6.2 M02 输入与资产

M02-01 模型导入

当前算法/规则: 正式支持 GLB/GLTF, 实验支持 FBX/OBJ; 按扩展名路由 loader, 生成统一 LoadedModel
输入 → 输出: File/URL → Three Group + 元数据
主要代码: engine/loaders/loadModelFromFile.ts

M02-02 FBX 兼容

当前算法/规则: 修复旧嵌入纹理文件名, 读取 ASCII/Binary Visibility 与 UnitScaleFactor, 恢复隐藏状态
输入 → 输出: FBX bytes → 修复后的模型
主要代码: loadFbxModel.ts、fbxVisibility.ts

M02-03 模型归一化

当前算法/规则: 变换父 Group, 不改 mesh 顶点; 计算包围盒、居中、落地、相机适配
输入 → 输出: Group → 规范化场景对象
主要代码: engine/scene/normalizeImportedModel.ts

M02-04 多模型放置

当前算法/规则: 新模型按已有场景包围盒并排放置, 保留每个对象独立 ID 与 transform
输入 → 输出: 多个 Group → 非重叠布局
主要代码: placeImportedModelBesideScene.ts

M02-05 参考图预处理

当前算法/规则: Atlas 请求总限制 4MB, 预留 512KB; 超限图像压缩后生成安全 Data URL
输入 → 输出: ReferenceImage → PreparedReference
主要代码: referenceImagePreprocessor.ts

M02-06 单/多视图配对

当前算法/规则: 用 referenceGroupId、role、derivedFrom 记录上传单图及生成多视图的稳定关系
输入 → 输出: 单图/多图 → reference group
主要代码: types/project.ts、GeneratePanel.tsx

6.3 M03 场景、相机与捕获

M03-01 相机快照

当前算法/参数: 保存 position/quaternion/target/near/far/fov/zoom/projectionMatrix/matrixWorld/viewMatrix/aspect
输入 → 输出: 活动相机 → SerializedCamera
主要代码: ProjectionCamera.ts、capture types

M03-02 适配捕获

当前算法/参数: current 保持用户视角; fit-object 按目标可见包围盒与 fill ratio 创建独立相机
输入 → 输出: 对象/方向/比例 → 捕获相机
主要代码: captureCurrentView.ts

M03-03 Color 捕获

当前算法/参数: 线性中间 RT + OutputPass 复现显示变换; 目标可用 viewport/clay/target-only/flat 模式
输入 → 输出: 场景 → PNG URL
主要代码: captureColor.ts、renderTargetUtils.ts

M03-04 Mask 捕获

当前算法/参数: 仅目标 mesh 可见并替换为白色 BasicMaterial, 黑色背景; 有效性读 RGB 亮度
输入 → 输出: 对象 ID → RGB mask
主要代码: captureMask.ts

M03-05 Depth 捕获

当前算法/参数: 线性相机深度 $(-viewZ - near) / (far - near)$, packDepthToRGB, alpha 固定 1
输入 → 输出: 对象/相机 → linear-view depth PNG
主要代码: captureDepth.ts

M03-06 Normal 捕获

当前算法/参数: 默认 view normal; 缩略图可 world normal; 编码为 $n * 0.5 + 0.5$
输入 → 输出: 对象/空间 → normal PNG
主要代码: captureNormal.ts

M03-07 GPU 读回

当前算法/参数: 大捕获可 tile 化; 每批等待 GPU fence、恢复共享 renderer, Worker 做 Y 翻转与 PNG 编码
输入 → 输出: RT pixels → Blob URL
主要代码: renderTargetUtils.ts、gpuReadbackPngWorker.ts

M03-08 分辨率保护

当前算法/参数: 通用完整捕获最高 2048; local repaint/depth/normal 交互预览通常为 1024; 请求分辨率不等于最终 UV 分辨率
输入 → 输出: 请求尺寸 → 受控捕获尺寸
主要代码: captureCurrentView.ts

6.4 M04 生成编排

M04-01 Texture Map 提示

当前算法/规则: 固定模型、相机、轮廓和构图; 用户提示只作为材质补充
输入 → 输出: 用户提示 → 约束提示
主要代码: GeneratePanel.tsx

M04-02 单视图生成

当前算法/规则: 当前相机捕获 + 多视图材质参考, 提交一个 single-view job, 成功后自动投影
输入 → 输出: 1 Capture → 1 Generation/Layer
主要代码: GeneratePanel.tsx、liclickApiClient.ts

M04-03 多视图生成

当前算法/规则: N 个相机各提交一个 single-view job, 共享 batch ID; 默认预设可为 6/10/14/自定义
输入 → 输出: N Capture → N Generation/Layer
主要代码: GeneratePanel.tsx

M04-04 参考图补全

当前算法/规则: 没有多视图参考时先生成一张多视图图, 并与单图建立 durable pair
输入 → 输出: 单参考 → 多视图参考
主要代码: GeneratePanel.tsx

M04-05 任务身份

当前算法/规则: clientGenerationId/serverJobId/taskId 归一化; 恢复时合并同一任务的记录, 避免重复 running 行

输入 → 输出: 任务事件 → 唯一 Generation

主要代码: generationIdentity.ts、generationStore.ts

M04-06 轮询与取消

当前算法/规则: 前台约 3.5/5 秒轮询, 任务最长等待 30 分钟; 支持 Abort 和服务端 cancel

输入 → 输出: Job ID → terminal state

主要代码: generation clients、GeneratePanel.tsx

M04-07 批量投影提交

当前算法/规则: 完成视角通过串行 commit queue 写资产与 Layer; 整批期间保留旧预览材质, 最终一次发布新栈

输入 → 输出: completed Generation → durable Layers

主要代码: GeneratePanel.tsx、layerStore.ts

当前风险: 主要编排仍集中在约 4000 行的 GeneratePanel.tsx。目标是逐用例迁入 M04/application, 面板只保留输入和进度显示。

6.5 M05 图层领域

M05-01 图层类型

当前规则: uv、projected、patch、normal; 角色另有 base/merged/repaint/content-aware

输入 → 输出: Layer contract → 可持久化层

主要代码: types/layer.ts

M05-02 新增空层

当前规则: 空 UV 层 imageUrl='', 绑定选中对象, 默认 visible/opacity/strength=1、normal blend

输入 → 输出: 可选名称/对象 → Layer

主要代码: layerStore.addEmptyLayer

M05-03 新增 UV 层

当前规则: 保存图像、对象、角色和可选 bakedTextureId; 新层插到栈顶

输入 → 输出: image URL → UV Layer

主要代码: layerStore.addUvLayer

M05-04 新增投影层

当前规则: 从 Generation/Capture 保存图像、相机、深度、对象矩阵; 普通生成默认不使用 capture silhouette mask

输入 → 输出: Generation + Capture → Projected Layer

主要代码: addProjectedLayerFromGeneration

M05-05 排序与显隐

当前规则: 数组 index 是显示顺序, 写回连续 order; 移动/重排使已烘焙投影标记 needsRebake

输入 → 输出: layer action → 新栈

主要代码: layerStore.ts

M05-06 参数调整

当前规则: opacity、strength、blendMode、HSV(hue/saturation/value); 影响烘焙结果时标记 rebake
输入 → 输出: 参数 → Layer patch
主要代码: layerStore.ts、projection/bake shader

M05-07 UV 合并发布

当前规则: 隐藏被消费源层; 写入指定空白 UV 层或在最前源位置插入新层; 记录 uvMergeVersion
输入 → 输出: source IDs + image → merged Layer
主要代码: mergeLayersIntoUvLayer

M05-08 保底空层

当前规则: 删除后若活动对象没有任何图层, 自动新增空 UV 层
输入 → 输出: 删除 → 有效图层栈
主要代码: ensureSelectedObjectHasLayer

6.6 M06 实时投影与可见性

M06-01 矩阵投影

当前算法/参数: clip = Projection_capture * View_capture * (M_capture * inverse(M_current)) * world_current
输入 → 输出: 世界点 → image UV/depth
主要代码: projectionMath.ts、ProjectedLayerMaterial.ts

M06-02 视锥门控

当前算法/参数: w>0 且 NDC x/y/z 均在 [-1,1]
输入 → 输出: projected point → inside
主要代码: projection shader

M06-03 源与蒙版

当前算法/参数: source alpha 或 ignoreSourceAlpha; mask 用 0.299R+0.587G+0.114B 再乘 mask alpha, 保留连续 feather
输入 → 输出: image/mask → source coverage
主要代码: projection shader

M06-04 面向门控

当前算法/参数: 无深度时硬拒绝 N·V<-0.35; 普通覆盖角范围 -0.62→-0.18; 有深度时使用绝对面向并采用 0.02→0.38
输入 → 输出: normal/view → angle coverage
主要代码: ProjectedLayerMaterial.ts

M06-05 深度可见性

当前算法/参数: linear-view 深度基准容差 max(0.00625, viewDepth*0.00075); 掠射角最多放大 5 倍; 中心加 8 邻域支持
输入 → 输出: captured/projected depth → visibility
主要代码: projection shader

M06-06 法线可见性

当前算法/参数: 捕获法线一致度从 0.72 到 0.92 平滑; surface-locked repaint 以深度为主, 避免低模相邻面条纹
输入 → 输出: normals → normal visibility
主要代码: projection shader

M06-07 覆盖率

当前算法/参数: $\text{opacity} \times \text{sourceAlpha} \times \text{angle} \times \text{visibility} \times \text{facing} \times \text{edgeFade}$; 图像边缘 0.015 内平滑

输入 → 输出: 各门控 → coverage

主要代码: projection shader

M06-08 质量分数

当前算法/参数: $\text{coverage} \times \text{depthWeight} \times \text{angleWeight} \times \text{qualityEdge}$; 角度 $\text{gamma} = 4 / \text{clamp}(\text{strength}, .25, 3)$; 质量边缘 0.035

输入 → 输出: coverage/geometry → quality

主要代码: projection shader

M06-09 多层实时合成

当前算法/参数: 每 texel 保留 Top-3 候选, 颜色一致性抑制离群视角, 质量主权重加 20% coverage 残差, 明显胜者向第一名收敛

输入 → 输出: N layers → preview color

主要代码: ProjectedLayerMaterial.ts、ProjectedLayerPreviewCompositor.ts

M06-10 纹理驻留

当前算法/参数: 小栈可直接 samplers, 大栈用 WebGL2 texture arrays; 按颜色/Mask/Depth/Normal 内存预算缩放预览并分条上传

输入 → 输出: layer assets → resident GPU textures

主要代码: ProjectedLayerMaterial.ts、preview cache

实时预览参数与 UV GPU 烘焙参数必须一起评审。修改 M06-04 至 M06-09 时, 变更范围自动升级为高风险算法变更。

6.7 M07 UV 烘焙、权重与合并

生产算法

M07-01 UV 栅格化

当前算法/参数: GPU 默认; 按 UV 顶点把 mesh 三角形画到方形 RT, 重建世界位置/法线并执行与实时投影同类门控

主要代码: gpuUvBakeRenderer.ts

M07-02 CPU 参考

当前算法/参数: 每 texel 4 个子采样点, 用重心坐标插值; 作为诊断/回退, 不保证所有容差与 GPU 完全相同

主要代码: uvRasterizer.ts

M07-03 候选选择

当前算法/参数: 每 texel 保留质量最高的 3 个。入选分数至少为 $\max(\text{quality}, \text{coverage} \times 0.08)$, coverage 阈值约 0.02/0.025

主要代码: bake renderer + quality worker

M07-04 颜色一致性

当前算法/参数: 在线性 RGB 中先求质量加权基色; 每候选质量乘 $0.35 + 0.65 \times \exp(-\text{distance}^2 / 0.22^2)$

主要代码: qualityBlend.worker.ts

M07-05 多视图权重

当前算法/参数: 强权重 $\text{quality}^2.4 / \text{sum}$, 软权重 $\text{coverage} / \text{sum}$, 最终 $0.8 \times \text{strong} + 0.2 \times \text{soft}$

主要代码: quality worker、bakeProjectedLayerToTexture.ts

M07-06 胜者收敛

当前算法/参数: 第一/第二质量比从 1.45 到 2.6, 质量差从 0.05 到 0.2, 两个 smoothstep 相乘后把混合色拉向第一名

主要代码: quality worker

M07-07 覆盖置信度

当前算法/参数: 内容补缝分析时 $\alpha = 1 - \Pi(1 - \text{coverage}_i)$; 普通 UV 输出写为不透明覆盖

主要代码: quality worker

M07-08 Overlay

当前算法/参数: 普通 Overlay 使用 $\text{coverage} * (0.75 + 0.25 * \text{qualityFade})$; 局部重绘 literal overlay 直接使用 coverage

主要代码: projectedOverlayComposition.ts

M07-09 后处理

当前算法/参数: gutter、拓扑内 gap、封闭 hole、物理 seam reconciliation、可选锐化; 均不得越过不相关 UV island

主要代码: dilation.ts、uvSeamReconciliation.ts

M07-10 GPU/CPU 编排

当前算法/参数: GPU 采样每层 raster, Worker/WebGPU 解析质量; 失败可回 CPU; 不静默降低调用方分辨率

主要代码: bakeProjectedLayerToTexture.ts

M07-11 合并 UV

当前算法/参数: 投影结果在前, 内容修补是 underlay; 当前合并只保存作者色 BaseColor。PBR 环境、曝光和灯光只参与视口显示, 不会被合并进 UV。旧字段 renderedColor/renderedColorMaskUrl 仍为兼容元数据, 不能据此把当前合并描述成“烘焙 PBR”

主要代码: mergeUvComposition.ts、EditorPage.tsx

M07-12 缓存签名

当前算法/参数: 对象、层 ID/资产/参数/顺序/分辨率/用途共同生成签名; 只复用精确匹配结果, 并合并并发相同 bake

主要代码: layerStackCache.ts、Editor cache

当前必须保留的已知差异

- GPU 是生产路径, CPU 是参考/回退路径; 两者当前部分角度和深度常量不同。任何“统一参数”都必须单独立项并运行 CPU/GPU 对比, 不能在普通 Bug 修复中顺手修改。
- texture.flipY=false 是现有 Three/glTF 约定。GPU 投影源的 Y 变换发生在采样坐标, 不允许为了修方向翻转最终 UV 图。
- 当前只处理单一 uv attribute、BaseColor RGBA、单活动对象, 不支持 UDIM。

6.8 M08 局部重绘

M08-A01 表面画笔选择

当前算法/规则: 指针笔压控制笔刷; 多相机画笔投影累积, 提交前从冻结相机重投影为一个 mask PNG

主要代码: ViewportCanvas.tsx、LocalRepaintDialog.tsx

M08-A02 ModelView 输入

当前算法/规则: 方形 1K 白模、最近纹理生成对应的多视图材质参考、方形 1K Flat 当前贴图; 本地画笔 mask 不送远端

主要代码: GeneratePanel.tsx、modelview client/service

M08-A03 深度保护

当前算法/规则: 同一冻结相机并行捕获 1K linear-view depth; 失败时生成结果仍可返回, 但回贴保护能力下降并记录 warning

主要代码: GeneratePanel.tsx

M08-A04 结果回贴

当前算法/规则: 使用 capture 相机/对象矩阵、累计画笔 mask、ignoreSourceAlpha=true、surface-locked-v1 和 depth 投回前表面
主要代码: ViewportCanvas.tsx、projection shader

M08-A05 边缘融合 v2

当前算法/规则: 只改蒙版内部边界 RGB, 不改 alpha; blend width 为短边约 1.8%, 限制 8-32px 且不超过蒙版内部距离的 1/3
主要代码: seamHarmonizationCore.ts

M08-A06 频带融合

当前算法/规则: 参考/生成分别做中低频 box blur, 边缘从参考低/中/细频带平滑过渡到生成; Lab 全局色彩匹配默认关闭
主要代码: seamHarmonizationCore.ts

M08-A07 持久化

当前算法/规则: 保存 raw result、可选 harmonized result、mask、viewport reference、seam version; 旧项目可回 legacy 模式
主要代码: EditorPage.tsx、Layer fields

M08-A08 当前提交策略

当前算法/规则: LOCAL_REPAINT_INTERACTIVE_UV_BAKE_ENABLED=false: 交互结束后不立即改 UV, 而是持久化一个可见 projected + local-repaint-overlay 层, 并保留隐藏 uv + local-repaint-draft 目标; 合并/导出时才按 literal overlay 规则烘焙
主要代码: ViewportCanvas.tsx、M07

M08-A09 禁用分支

当前算法/规则: 代码保留“每次笔画后直接烘焙 UV”的兼容分支, 但当前常量关闭。它不是截图所示版本的生产行为, 修改或启用必须单独立项并重跑重绘、UV 与导出回归
主要代码: ViewportCanvas.tsx

M08-B01 Edit/Repair mask

当前算法/规则: edit=max(user,hole), 可膨胀; protect=object && !edit; 支持小连通域过滤、羽化、ROI 裁剪
主要代码: maskUtils.ts

M08-B02 Provider 编辑

当前算法/规则: 构造约束提示、上传 image/mask/protect/reference, 轮询或取消远端 edit job
主要代码: imageEditProvider.ts、EditorPage.tsx

M08-B03 本地合并

当前算法/规则: 保护未选区域, 按 feather mask 合成原图和编辑图; edit_layer 更新原图, repair_current_view 生成 UV repair layer
主要代码: imageUtils.ts、acceptLocalRepaint

6.9 M09 内容识别补缝

ID	当前算法/参数	主要代码
M09-01 投影基图	普通 projected layers 确定性排序, 2K 上限, transparent + coverage confidence, 不做 dilation/PNG	executeContentAwareRepair、M07
M09-02 拓扑构建	保守三角栅格、strict core、union-find surface component/region、冲突标记; 位置量化 100000, UV epsilon 1e-7	buildSurfaceTopology.ts
M09-03 物理接缝	按几何边匹配 UV 两侧, 默认 seam band 1px、normal dot 至少 0.72; 建立显式像素 link	buildSurfaceTopology.ts
M09-04 缺口识别	hard alpha 使用实时空覆盖边界, weak 阈值 64、增长 1px; 按分辨率过滤小噪点并保留长窄 seam	buildRepairMask.ts、policy
M09-05 源排除	缺口和周边 2-4px 不可作 donor; donor alpha 至少 64; 颜色离群阈值 64	surfaceAwareRepair.ts、policy
M09-06 表面传播	4 邻域 BFS, 距离 64-128px; 同 region 内传播, 最多跨一个物理 seam; 锁定主 donor region, 阈值 18	surfaceAwareRepair.ts
M09-07 不可达策略	fillUnreachableWithGlobalAverage=false, 没有局部证据就保持透明	policy + repair core
M09-08 输出边缘	成功修复 texel alpha 255; 1px coverage skirt 只填近空区域; 4px RGB bleed alpha 仍为 0	surfaceAwareRepair.ts
M09-09 Worker/缓存	拓扑缓存、最多 4 个 persistent projection cache; repair 默认独立 Worker, 支持 Abort/进度与 buffer transfer	content-aware runtime/cache
M09-10 原子发布	稀疏 PNG 完成、解码、分条 GPU 上传后才把可见图层一次性写入 store	EditorPage.tsx

6.10 M10 生产流程

ID	当前规则	主要代码
M10-01 Pipeline revision	stage=texture/retopology/uv/bake; revision 只追加不原地修改, 输入/输出资产 immutable	types/project.ts、projectPipeline.ts
M10-02 下游失效	上游重跑时以 staleRevisionIds 覆盖标记所有下游旧 revision, 保留历史	markDownstreamPipelineRevisionsStale
M10-03 Auto UV/Retopo	浏览器上传资产, 主服务代理远端任务, 保存 owner/history/artifact, 完成后进入下一 stage	asset processing routes/services
M10-04 PBR Bake	高/低模、颜色、Cage、匹配和通道设置提交远端 Substance bake; 浏览器负责工作区、检查和发布	Bake workspace + substanceBakeService.ts
M10-05 烘焙通道	BaseColor、Normal、AO、Curvature、World Normal、Thickness、Position、可选 Roughness/Metallic	Bake contracts/settings

远端 UV、拓扑和 Substance 算法本体不在本仓库。这里的可维护范围是输入契约、任务状态、代理、产物选择、恢复和流水线交接, 不能把远端黑盒行为写成浏览器本地算法。

6.11 M11 输出与集成

ID	当前规则	主要代码
M11-01 BaseColor 准备	收集可见 UV/投影/repair 栈，精确缓存命中则复用，否则按需 UV bake；局部重绘导出时修复透明/方向语义	texturedExportUtils.ts
M11-02 模型导出	Scene/对象支持 GLB、FBX、OBJ、STL；纹理材质统一命名，FBX 有自建 binary 写出路径	engine/export
M11-03 图像/视频	BaseColor PNG、已有 normalMap PNG、Viewport PNG、5 秒 WebM turntable	export modules
M11-04 工程包	服务端导出工程 package；.liclick3d 便携 zip 仍未正式交付	server export service
M11-05 Photoshop	主服务 bridge + UXP 插件维护会话和图像交换	photoshopBridgeService.ts、integration
M11-06 DCC	Blender/3ds Max connector 当前仍是占位；不能把验证脚本描述成实时连接器	connectors、validation scripts

6.12 M12 状态、历史与版本

ID	当前真值	用途
M12-01 编辑器撤销	最多 80 步，快照只包含 objects/layers；runtime step 可提供 undo/redo；当前持久化函数是 no-op，刷新后不保证恢复	用户会话内撤销
M12-02 工程 autosave	每次成功保存后最多 5 份完整 JSON	用户工程灾难恢复
M12-03 生成/任务历史	Generation 记录和服务端 UV/Retopo/Bake history	远端任务恢复与产物下载
M12-04 Pipeline revision	工程内 append-only stage checkpoints	生产资产血缘，不是代码历史
M12-05 Git	分支、提交、tag、CHANGELOG.md、docs/changes	唯一的源码/算法修改历史
M12-06 ADR	docs/decisions 中不可变决策记录	解释为什么选择某个架构/算法

不得混淆这六种历史。尤其不能用编辑器撤销或项目 autosave 代替 Git。

6.13 M13 平台与性能基础设施

ID	当前责任	主要代码
M13-01 身份边界	主服务持有 Web/飞书会话；本地组件持有个人 Liclick/Atlas 身份；本地调用带主服务签发的 identity proof	auth/services
M13-02 Transport	根据 provider status 选择 workspace Atlas 或 personal local component，不允许 UI 私自选 endpoint	liclickTransport.ts、auth strategy
M13-03 Heavy task	对昂贵 capture/bake/repair 排队、取消、首结果和交互预算管理	heavyTaskScheduler.ts
M13-04 Frame budget	交互期间 Worker/GPU 上传/readback 分片，让可见视口优先呈现	performance modules/workers
M13-05 Cache	Blob URL 注册、预览纹理、layer-stack bake、content-aware topology/projection 等按明确签名缓存	cache modules
M13-06 Telemetry	记录模块 start/complete/fail、性能分解和身份聚合；不得成为业务真源	telemetry clients/services

7. 版本注册表

版本对象	当前值	何时升级
仓库产品版本	根 package.json 为 0.1.13	对外发布时按 SemVer
Web/Server package	当前各为 0.1.3，与根版本不一致，属于待治理项	统一发布流程建立后同步
工作区格式	workspaceVersion=0.6.0	Project 持久化兼容性变化
Project pipeline	version=1	revision 结构不兼容变化
UV merge composition	UV_MERGE_COMPOSITION_VERSION=4	合并颜色/光照/顺序语义变化
Local repaint seam	metadata seamHarmonizationVersion=2	边缘融合输出发生可见变化
ModelView workflow	2026.08.17-a9dbbca-flux2-klein-truev3-3input-r2	远端 workflow 或三输入契约变化
Content-aware projection cache	liclick-content-aware-projection-v1	缓存输入签名或像素语义变化
Editor history storage key	liclick-editor-history-v1，但当前不实际持久化	恢复格式真正启用时

7.1 算法名称与规范版本

规范版本描述本文档中的输入、公式、输出和回退契约；运行版本是代码或工程文件中真实存在的兼容标记。两者不能互相冒充。此次盘点为算法规范建立 1.0.0 基线，但不向旧工程补写不存在的运行字段。

ALG-PROJ-001
基础名称： 捕获锁定重投影（Capture-Locked Reprojection）
规范版本： 1.0.0
当前运行版本/标记： 未独立持久化；随 Layer camera/object matrix 契约
兼容性说明： 改矩阵顺序或坐标空间为 Major
ALG-PROJ-002
基础名称： 连续覆盖门控（Continuous Coverage Gate）
规范版本： 1.0.0
当前运行版本/标记： 未独立持久化
兼容性说明： 改 alpha/mask/opacity 语义为 Major
ALG-PROJ-003
基础名称： 深度-法线表面可见性（Depth-Normal Surface Visibility）
规范版本： 1.0.0
当前运行版本/标记： projectionVisibilityPolicy 有 standard/surface-locked-v1
兼容性说明： 改默认阈值至少 Minor；旧 Layer 可见结果变化时为 Major
ALG-PROJ-004
基础名称： Top-3 一致性合成（Top-3 Consistency Blend）
规范版本： 1.0.0
当前运行版本/标记： 未独立持久化
兼容性说明： 改候选数、权重或颜色距离为 Major

ALG-PROJ-005
基础名称: 有序覆盖合成 (Ordered Literal Overlay) 规范版本: 1.0.0 当前运行版本/标记: 由 blendMode/role 选择 兼容性说明: 改 local repaint literal 语义为 Major
ALG-PROJ-006
基础名称: 纹理驻留预算 (Resident Texture Budgeting) 规范版本: 1.0.0 当前运行版本/标记: 未持久化, 只影响 runtime 兼容性说明: 只改预算且像素不变通常为 Patch/Minor
ALG-UV-001
基础名称: 合并输入冻结与分类 (Merge Input Freeze & Classification) 规范版本: 1.0.0 当前运行版本/标记: UV_MERGE_COMPOSITION_VERSION=4 兼容性说明: 源层集合、顺序或角色变化为 Major
ALG-UV-002
基础名称: UV0 三角投影栅格 (UV0 Triangle Projection Raster) 规范版本: 1.0.0 当前运行版本/标记: merge version 4 间接覆盖 兼容性说明: 改 UV 集、方向或采样位置为 Major
ALG-UV-003
基础名称: Runtime 可见性捕获 (Runtime Visibility Refresh) 规范版本: 1.0.0 当前运行版本/标记: 无独立字段 兼容性说明: 改 capture 复用/刷新判定至少 Minor
ALG-UV-004
基础名称: Top-3 质量合成 (Top-3 Quality Composite) 规范版本: 1.0.0 当前运行版本/标记: merge version 4 间接覆盖 兼容性说明: 改权重/颜色空间为 Major
ALG-UV-005
基础名称: Literal Overlay 批处理 (Literal Overlay Batch) 规范版本: 1.0.0 当前运行版本/标记: merge version 4 间接覆盖 兼容性说明: 与逐层结果不一致视为 Bug

ALG-UV-006
基础名称: 拓扑约束后处理 (Topology-Constrained Postprocess) 规范版本: 1.0.0 当前运行版本/标记: 调用参数 + cache signature 兼容性说明: 跨 UV island 的规则变化为 Major
ALG-UV-007
基础名称: Under 合成 (Under Composite) 规范版本: 1.0.0 当前运行版本/标记: merge version 4 兼容性说明: 改 under/over 关系为 Major
ALG-UV-008
基础名称: Straight-RGBA 发布 (Straight-RGBA Publish) 规范版本: 1.0.0 当前运行版本/标记: merge version 4 兼容性说明: 改为预乘 alpha 或翻转方向为 Major
ALG-LR-001
基础名称: 冻结视角选择重投影 (Frozen-View Selection Reprojection) 规范版本: 1.0.0 当前运行版本/标记: archived mask + capture camera 兼容性说明: 改 aspect/mask 空间为 Major
ALG-LR-002
基础名称: 三输入 ModelView 生成 (Three-Input ModelView Generation) 规范版本: 1.0.0 当前运行版本/标记: workflow 2026.08.17-a9dbbca-flux2-klein-truev3-3input-r2 兼容性说明: 输入数、顺序或服务语义变化必须换 workflow 标记
ALG-LR-003
基础名称: 并行深度保护 (Parallel Depth Guard Capture) 规范版本: 1.0.0 当前运行版本/标记: depthEncoding=linear-view 兼容性说明: 改编码/相机为 Major
ALG-LR-004
基础名称: 边界谐调 (Boundary-Only Seam Harmonization) 规范版本: 1.0.0 当前运行版本/标记: seamHarmonizationVersion=2 兼容性说明: 可见像素算法变化升级 seam version

ALG-LR-005
基础名称: 表面画笔重投影 (Surface Brush Reprojection) 规范版本: 1.0.0 当前运行版本/标记: live mask + minimum facing 0.08 兼容性说明: 改笔刷空间/面向门控至少 Minor
ALG-LR-006
基础名称: 低延迟实时覆盖 (Low-Latency Live Overlay) 规范版本: 1.0.0 当前运行版本/标记: surface-locked-v1 兼容性说明: 与最终持久化不一致视为 Bug
ALG-LR-007
基础名称: 延迟投影持久化 (Deferred Projected Persistence) 规范版本: 1.0.0 当前运行版本/标记: interactive UV flag=false 兼容性说明: 改默认提交目标为 Major
ALG-LR-008
基础名称: 即时 UV 分支 (Interactive UV Commit, 禁用) 规范版本: 1.0.0-disabled 当前运行版本/标记: feature constant=false 兼容性说明: 启用前必须新 Minor/Major 版本和迁移说明
ALG-LR-009
基础名称: Provider 兼容编辑 (Provider Image-Edit Compatibility) 规范版本: 1.0.0-compat 当前运行版本/标记: provider/workflow 各自任务标记 兼容性说明: 与主路径合并需 ADR

7.2 当前版本治理缺口

- ALG-PROJ-* 的核心阈值尚无独立的 projectionAlgorithmVersion 持久化字段；旧工程重开会使用新代码参数重新显示。这是明确的 P1 治理债务。
 - UV 多个算法目前共同依赖 UV_MERGE_COMPOSITION_VERSION，尚未拆成 raster/composite/postprocess 子版本；修改时必须在变更记录逐项判断影响。
 - runtime feature constant 不等于版本。false → true 的功能开放必须有变更单、规范版本和旧工程策略。
 - 规范版本只在本文件和变更记录中递增；运行版本只有在代码真实写入并有迁移/回退时才能登记为“已持久化”。
- 算法变更必须判断版本类型：
- Patch：只修实现错误，输入输出语义不变。
 - Minor：新增可选能力或参数，旧工程和默认输出兼容。
 - Major：默认像素语义、Layer/Project 字段、工作区格式或跨模块契约不兼容。
- 不能只改注释中的版本。版本字段、迁移/回退、测试基线和本文档必须一起更新。

8. 变更分级与 Codex 修改上限

等级	典型范围	必须条件
L0 文档/文案	不改变运行行为	自检链接和格式
L1 模块内修复	单模块、通常不超过 5 个源文件或 300 行净变化	变更记录、定向测试、另一人 review
L2 算法/跨层变化	像素、权重、性能、持久化或 2 个模块	先批准方案、前后样例/指标、完整回归、算法版本判断
L3 架构/数据迁移	跨多个模块、目录重构、schema/security/release	ADR、分阶段实施、迁移与回滚、两人明确批准

默认 Bug 修复授权只覆盖 L1。如果 Codex 发现需要满足以下任一条件，必须停在方案和证据阶段，不能直接扩大修改：

- 超过 5 个源文件或预计超过 300 行净变化；
- 修改两个以上大模块；
- 改 Project/Layer/Capture/Generation 持久化字段；
- 改投影/UV 权重、深度/法线阈值、颜色空间、方向或默认分辨率；
- 搬目录、拆超大文件、替换状态库/渲染器/服务协议；
- 删除兼容路径或需要迁移旧工程。

用户明确批准一个 L2/L3 方案后才能继续。批准的是列出的范围，不是无限制重写。

9. 标准变更流程

9.1 开始前

1. 同步主分支，查看 `git status`，列出并保护他人的未提交修改。
2. 建立分支：`codex/<change-id>-<short-name>` 或 `dev/<change-id>-<short-name>`。
3. 从 `docs/changes/CHANGE_TEMPLATE.md` 创建变更记录。
4. 填写：问题证据、期望、主模块 ID、影响模块、计划文件、明确不做什么、验收和回退。
5. L2/L3 先由另一人批准方案；架构决定另建 ADR。

9.2 实施中

1. 先添加能复现问题的测试或可重复基线。
2. 只修改计划文件；新增影响时先更新变更记录。
3. 不做机会主义重构，不混入格式化噪音。
4. 算法参数集中为命名常量；实时/GPU/CPU/Worker 的副本必须列在变更记录中。
5. 任何 fallback 都必须可观测，并保留原请求分辨率和错误原因。
6. 更新对应模块章节、版本注册表和 `CHANGELOG.md`。

9.3 完成前

1. 运行模块定向测试、`typecheck`，按风险运行 `build/lint/回归`。

- 2. 记录命令、结果、视觉样例、性能前后值和未覆盖项。
- 3. 检查 diff 只包含本次范围，检查是否误带工作区资产、secret、dist 或他人改动。
- 4. 由非实现者 review；高风险像素算法必须实际看前后输出。
- 5. 使用聚焦提交并在变更记录中回填 commit；合并后更新状态为 Released/Merged。

10. Git 与提交规范

- 一个问题一个分支；不要用“个人长期功能分支”堆积多个无关问题。
- 提交格式：type(module-id): 中文结果，例如 fix(M06): 修复掠射角深度断层。
- type 使用 fix、feat、refactor、perf、test、docs、chore。
- 合并前允许多个清晰提交；禁止 update、try again、final 这类无信息提交。
- 发布 tag 使用根产品版本；变更记录引用 tag 和最终 commit。
- CHANGELOG.md 只写用户或维护者可感知结果，技术证据放 docs/changes。
- 历史文档和 ADR 合并后只追加勘误，不改写原决定；被替代时新建记录并链接 superseded ID。

两人并行工作建议：一人是 Owner，一人是 Reviewer。高风险修改不要由同一个人自写、自测、自合并。

11. 模块验收矩阵

变更模块	必跑或最低验证
M01/M12	server typecheck/build、pipeline persistence、editor project route load、手工保存/重开
M02/M03	model import progress、FBX legacy repair、placement/camera、捕获人工对照
M04	generation polling、local runtime health、取消/恢复、单/多视图至少各一次
M05/M06	projection layers、eye toggle、相机旋转、对象 transform 后对齐
M07	projection tests、UV composite backpressure、model export orientation、CPU/GPU compare、目标分辨率
M08	local repaint seam、bake batching、画笔边缘/暗部/多相机/刷新恢复
M09	test:content-aware-repair、完整率、断开 UV island 不串色、取消
M10	bake entry/base-color selection、server bake artifact/pipeline、对应 smoke
M11	GLB/FBX/OBJ/BaseColor 方向和重开验证、Photoshop smoke（如涉及）
M13	heavy task scheduler、frame budget、performance lab、压力场景

pnpm typecheck 是基础门，不代表像素算法正确。像素算法必须有输出证据或 golden/统计对比。

12. 当前架构债务与拆分顺序

P0

- 现状：ViewportCanvas.tsx 约 1.3 万行
- 风险：画笔、捕获、UV commit、基准互相牵连
- 建议处理：先提取 repaint selection、capture adapter、UV commit application service

P0

现状: `EditorPage.tsx` 约 7 千行

风险: 保存、合并、内容修复、导出编排混合

建议处理: 按 M01/M07/M09/M11 逐用例抽离

P0

现状: `ProjectedLayerMaterial.ts` 约 5 千行

风险: Shader 生成、缓存、采样预算、显示材质耦合

建议处理: 先共享 projection policy/constants, 再拆 candidate/compose/resource modules

P0

现状: `GeneratePanel.tsx` 约 4 千行

风险: UI、任务状态、捕获、持久化和批处理耦合

建议处理: 抽 M04/application 单/多视图 use cases

P1

现状: CPU/GPU/preview 投影参数多处复制且存在差异

风险: 修一处可能三条路径不一致

建议处理: 建共享策略契约和校准测试, 不直接强行统一

P1

现状: 两条局部重绘路径并存

风险: 修错路径或重复逻辑

建议处理: 在产品确认后 ADR 决定主路径与退役计划

P1

现状: Project JSON 只做部分 Zod 校验, `packages/core` 不是当前真源

风险: 旧工程/服务端断言风险

建议处理: 建 schema/migration 模块后再迁移

P1

现状: 编辑撤销历史持久化函数为空

风险: 用户误以为刷新可恢复

建议处理: 明确 UI 文案或真正实现持久化

P2

现状: 根与 workspace package 版本不一致

风险: 发布和问题定位困难

建议处理: 建统一 release 脚本/检查

P2

现状: 历史文档按日期散落

风险: 容易引用过期方案

建议处理: 以本文档和变更记录索引, 不删除历史

拆分必须作为独立 `refactor` 变更, 先锁定行为测试, 再移动代码。不能借 Bug 修复一次性完成上述全部债务。

13. 文档维护规则

- 本文档只保留当前规则和当前算法，不记录逐日工作流水账。
- 每次修改在文末追加一行文档修订历史，并在 [docs/changes](#) 留完整证据。
- 文件/符号改名时更新主要代码位置；小模块 ID 不因文件移动而变化。
- 新增大模块必须经过 ADR；新增小模块必须分配 ID、责任、输入输出和最低测试。
- 参数表写生产默认值；实验 query/localStorage 开关单独标为诊断，不得写成用户功能。
- 尚未实现的目标必须明确标注“目标/计划”，不能出现在“当前实现”表中。

14. 编辑器界面模块总图

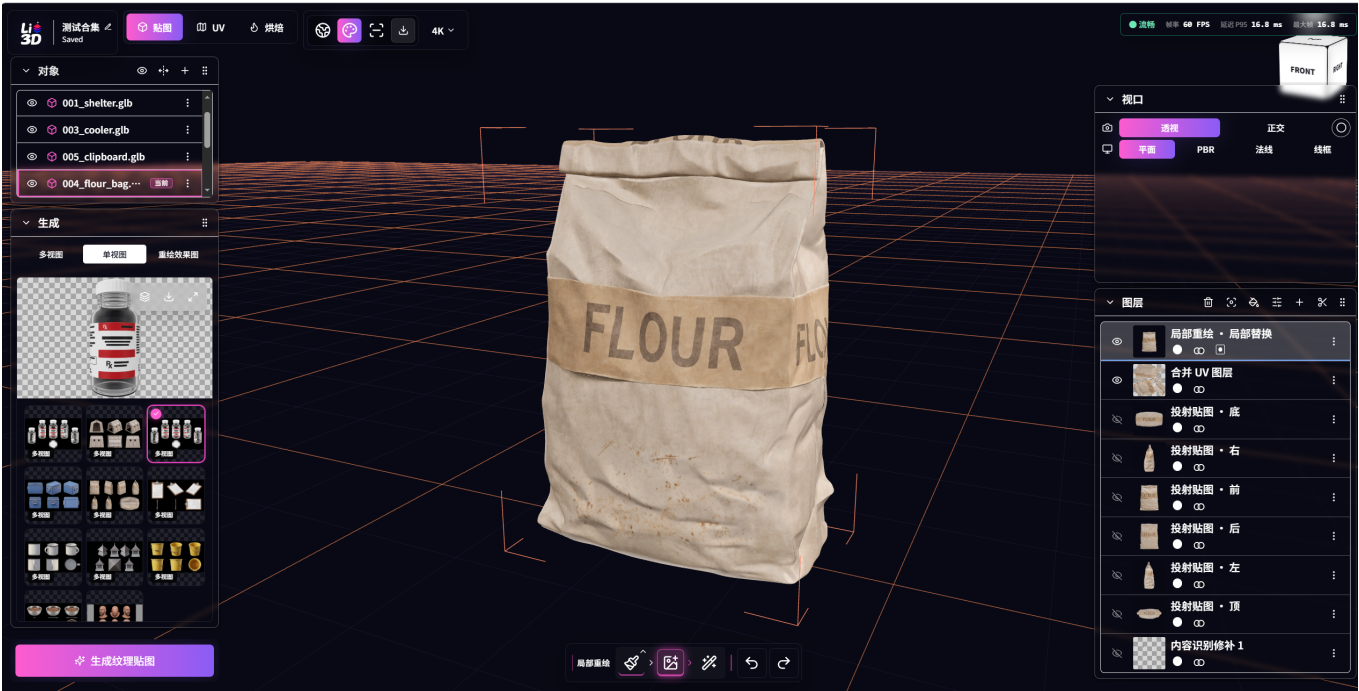


图 1 LI3D 贴图编辑器界面与 UI 模块审计基准

本节以贴图编辑器当前界面为第一层目录。界面名称只是入口；真正的维护边界由“模块 ID + 状态所有者 + 应用用例 + 算法 ID + 持久化字段”共同确定。

14.1 截图区域与唯一模块编号

UI-01 工程头
当前职责：返回工程列表、显示/修改工程名、显示 Saved/Saving/Failed
状态所有者：projectStore、Editor 本地保存状态
入口代码：EditorShell.tsx、EditorPage.tsx
调用的大模块：M01/M12
UI-02 生产流程切换
当前职责：贴图 → UV → 烘焙 页面/阶段导航；贴图页本身不执行远端 UV 或 Bake
状态所有者：route + Project pipeline
入口代码：WorkflowModuleSwitcher.tsx
调用的大模块：M10
UI-03 工作区与输出栏
当前职责：场景、贴图、法线工作区；导出菜单；1K/2K/4K/8K 设置
状态所有者：workspaceLayoutStore、sceneStore、settingsStore
入口代码：EditorShell.tsx
调用的大模块：M03/M10/M11/M12

UI-04 对象面板

当前职责：活动对象、显隐、重命名、聚焦、变换、复制、下载、删除、导入、自动排列
状态所有者：sceneStore + projectStore
入口代码：ObjectsPanel.tsx
调用的大模块：M02/M03/M05/M12

UI-05 生成面板

当前职责：多视图、单视图、重绘效果图；参考图选择；任务提交/恢复；生成后自动建投影层
状态所有者：generationStore、referenceStore、layerStore
入口代码：GeneratePanel.tsx
调用的大模块：M03/M04/M05/M08/M13

UI-06 中央视口

当前职责：Three 场景、相机、轨道控制、对象变换、实时投影材质、选择/画笔命中、局部重绘预览
状态所有者：sceneStore + GPU runtime
入口代码：ViewportCanvas.tsx、SceneRoot.tsx
调用的大模块：M02/M03/M06/M08/M13

UI-07 视口控制

当前职责：透视/正交、当前视图捕获、Flat/PBR/Normal/Wire、PBR 环境/曝光/灯光
状态所有者：sceneStore、settingsStore、generationStore.lastCapture
入口代码：ViewportPanel.tsx
调用的大模块：M03/M06

UI-08 视图立方体与性能 HUD

当前职责：固定相机方向；显示 FPS、P95、最大帧等诊断信息
状态所有者：camera runtime + performance runtime
入口代码：ViewCube/performance components
调用的大模块：M03/M13

UI-09 图层面板

当前职责：图层选择、显隐、顺序、透明度、叠加语义、调整、复制/替换/删除、内容修补、合并 UV
状态所有者：layerStore + Editor 用例
入口代码：LayersPanel.tsx、EditorPage.tsx
调用的大模块：M05/M06/M07/M08/M09/M12

UI-10 底部工具条

当前职责：场景选择/移动/旋转/缩放；贴图模式局部选择、局部生图、结果回贴；撤销/重做
状态所有者：sceneStore、editorHistoryStore
入口代码：BottomToolDock.tsx、ViewportCanvas.tsx
调用的大模块：M03/M08/M12

UI-11 工作区 Dock

当前职责：按当前 mode 过滤面板，按 order 排列，支持折叠、左右 Dock 和移动端抽屉

状态所有者：workspaceLayoutStore

入口代码：WorkspaceDock.tsx、EditorShell.tsx

调用的大模块：M12

UI-12 模态任务层

当前职责：参考导入、兼容局部重绘、长任务进度/取消、编辑锁

状态所有者：Editor task state

入口代码：EditorPage.tsx、dialogs

调用的大模块：M02/M04/M08/M09/M13

14.2 工作区切换的实际含义

场景/地球

立刻执行的状态变化：workspaceMode=scene，同时 displayMode=pbr

显示的面板：对象、对象变换、参考图、视口

不会执行的事：不烘焙 PBR，不修改图层像素

贴图/调色盘

立刻执行的状态变化：workspaceMode=texture，同时 displayMode=flat

显示的面板：对象、生成、视口、图层、局部工具

不会执行的事：不自动合并 UV

法线/框选样图标

立刻执行的状态变化：workspaceMode=normal，同时 displayMode=normal

显示的面板：法线预览与法线生成占位面板

不会执行的事：当前法线生成面板仍是 Coming Soon

下载

立刻执行的状态变化：打开导出菜单

显示的面板：不切换核心图层算法

不会执行的事：不等于 Ctrl+S 保存工程

1K/2K/4K/8K

立刻执行的状态变化：修改 settingsStore.resolution

显示的面板：影响后续捕获、合并、修补或导出的调用方分辨率

不会执行的事：不重采样已经存在的源图层

Flat/PBR/Normal/Wire 是视口显示模式，不是图层类型：Flat 显示作者色；PBR 在作者色上实时应用材质/环境/曝光/灯光；Normal 显示法线；Wire 显示线框。切换这些按钮不会重写 Layer PNG。

14.3 面板生命周期和编辑锁

- 1. EditorPage.panelDefinitions 是贴图编辑器面板注册表。
- 2. WorkspaceDock 仅呈现 visible=true 且 panel.mode=all/currentMode 的面板，再按 order 排序。
- 3. 面板折叠/可见性/左右位置属于工作区布局，不属于算法结果。

- 4. 内容修补或局部生图等长任务运行时，`editorTaskRunning`/对应 `lock` 会阻止会破坏输入快照的对象和图层操作。
- 5. UI 新按钮必须先登记 UI ID → M ID → Use Case/ALG ID；禁止在 `React onClick` 内直接复制一套像素算法。

15. 每个界面模块的操作调用表

15.1 UI-01/UI-02/UI-03 工程头、流程和设置

操作	调用链	结果/持久化
修改工程名	<code>EditorShell.commitProjectRename</code> → <code>EditorPage.handleRenameProject</code>	本地工作区项目走服务端 <code>rename/save</code> ；Project name 更新
Ctrl/Cmd+S	<code>shortcut project.save</code> → <code>handleManualSave</code> → <code>getProjectSnapshot</code> → <code>prepareProjectForWorkspaceSave</code> → <code>PUT /api/projects/:id</code>	详见第 20 节；JSON 清单与二进制资产均保存
点击 UV	<code>WorkflowModuleSwitcher.onOpenUv</code>	跳到 UV 工作区/路由；不等于右侧剪刀“合并 UV”
点击烘焙	<code>handleOpenBake</code> ，必要时发布当前阶段产物	进入 Bake workspace；不是把当前 PBR 画面截图保存
修改分辨率	<code>settingsStore.setResolution</code>	保存到 Project settings；只影响此后发起的分辨率敏感任务

15.2 UI-04 对象面板

操作	当前规则	联动范围
选择对象	设置 <code>selectedObjectId/activeObjectId</code> ，视口、图层和生成面板按对象过滤	M03/M05
眼睛	历史快照后修改 <code>SceneObject visibility</code> ，并同步 Project	捕获和导出只处理符合各自 <code>visible policy</code> 的对象
重命名	历史快照后同时修改 Scene 与 Project 对象名	不改模型资产文件名
复制	<code>clone runtime Group/材质/对象</code> 记录并分配新对象 ID	不复制旧对象的生成和图层血缘
下载对象	<code>GLTFExporter</code> ，binary GLB、 <code>embedImages</code> 、 <code>onlyVisible</code>	这是对象快照导出，不是完整贴图栈生产导出
删除	先记录历史，再删除对象关联 <code>Layers/Generations/Project object</code> ，并触发立即保存	是跨 M02/M05/M12 的事务，不能只删 Scene node
导入 +	文件路由支持模型与参考图扩展名；模型进入 <code>loader/归一化/并排放置</code>	资产先进入 runtime，保存时转为 workspace 相对路径

15.3 UI-05 生成面板

Tab/操作	当前算法	输出
多视图	N 个相机分别执行 capture + generateTextureSingleView; 共享 batch, 但每个视角有独立 Capture/Generation	成功一个视角就可创建一个普通 Projected Layer; 批次后可自动内容补缝
单视图	只使用当前/选定相机提交同一 single-view 协议	1 Capture + 1 Generation + 1 Projected Layer
只有单张参考	先生成/建立配对多视图参考, 再提交纹理任务	Reference group 记录 derived/source 关系
重绘效果图	展示主局部重绘 ModelView 结果并为第 3 步回贴提供来源	生成结果不是立刻写入 UV, 见第 19 节
生成纹理贴图	冻结请求所需输入、提交任务、轮询/取消/恢复、持久化结果、建层	不能在任务返回后重新读取已经变化的相机代替原 Capture

15.4 UI-06/UI-07/UI-08 视口

操作	当前规则	是否改变资产
旋转/缩放视口	OrbitControls 改当前相机	否; 只有捕获/生成时才冻结相机
视图立方体	设置标准方向与 target	否
透视/正交	sceneStore.projectionMode 选择相机模型	保存为工程/工作区状态; 已有 Layer 仍使用自己的 capture camera
圆形捕获按钮	captureCurrentView, 对象为当前选择, 分辨率取设置值, 结果写 generationStore.lastCapture	产生内存 Capture; 被生成/保存流程采用时才持久化
PBR 参数	环境 preset、曝光 0.7-1.8、环境强度 0-1.2、主光 0-2.4、方位 -180°-180°	只改显示设置, 不改 BaseColor PNG
FPS HUD	采样帧时间并显示 FPS/P95/max frame	诊断信息, 不是工程算法输入

15.5 UI-09 图层面板顶部动作

图标/动作	用例	当前结果
垃圾桶	清除当前对象图层	先隐藏、下一帧删除并保留 session undo; 不是删除对象
靶心	聚焦活动对象/图层对应模型	只改当前相机
内容识别修补	executeContentAwareRepair	新建/更新稀疏 uv + content-aware-underlay 层
调整	展开 LayerAdjustmentsPanel	Projected 可调 strength 25%-300%; 所有适用层可调 H/S/L -100-100
+	layerStore.addEmptyLayer	栈顶新增空白 UV 层, 默认 normal/opacity=1
剪刀	mergeLayersToUvLayer(visibleProjectedLayerIds)	执行完整 UV 合并事务, 见第 18 节
拖拽点阵	Dock/面板布局操作	不改变图层栈像素

15.6 UI-09 图层行操作

控件	当前实际行为	注意事项
眼睛	修改 visible；局部重绘会扩展到配对的 visible projected 层与 hidden draft	visible=false 不会删除资产
缩略图/名称	激活图层；菜单可编辑、替换、重绘、复制、合并、删除	hidden local-repaint-draft 不作为普通行显示
白色圆点	opacity 交互入口/指示	opacity 乘最终 coverage；不是 strength
链环	当前 UI 只在 normal □ overlay 间切换	类型声明虽有 multiply/screen/soft-light，但此按钮不提供它们
方框/蒙版标记	表示存在 mask 或特定局部来源	mask 的空间必须看 maskSpace=projection/uv，不能只看图标
上下拖动	改数组顺序并重写连续 order	会使相关 bake/cache 失效；order=0 是面板最上方

15.7 UI-10 局部三步工具条

- 1. **选择区域**：inpaint-add/inpaint-subtract 在模型表面累计选择；左键添加、右键移除，支持笔刷大小、清空与反选。
- 2. **局部生图**：冻结相机和对象矩阵，读取累计选择或建立全图 mask，再走 ALG-LR-001~004。
- 3. **应用重绘**：选择成功返回的结果，在模型表面用投影画笔决定实际回贴 coverage，走 ALG-LR-005~007。
- 4. **撤销/重做**：使用最多 80 步的 session history；当前刷新页面后不保证保留。
- 5. Projected eraser 的实现仍在代码中，但
PROJECTED_ERASER_TOOL_ENABLED=false，当前发布版不应在文档中写成可用功能。

16. 图层类型、角色和效果唯一判定

16.1 四个维度不能混用

维度	回答的问题	当前值
type	像素处于什么坐标空间/由哪个渲染器消费	uv/projected/patch/normal
role	该层在业务合成中扮演什么语义	base-color/merged-uv/local-repaint-draft/local-repaint-overlay/content-aware-underlay
blendMode	普通层还是有序覆盖层	schema 有 5 种; 当前图层行只切 normal/overlay
projectionVisibilityPolicy	投影怎样锁定可见表面	standard/surface-locked-v1

因此“局部重绘”不是一个 LayerType; 它通常是 type=projected + role=local-repaint-overlay + surface-locked-v1, 另配一个不显示的 type=uv + role=local-repaint-draft。

16.2 截图中各类图层的准确差别

局部重绘 · 局部替换
底层身份: 可见 projected, role=local-repaint-overlay 数据: 生成图、累计 brush mask、capture camera/object matrix、depth、seam 元数据 视觉效果/合成规定: 不参加普通 Top-3 平均; 在普通投影后按 coverage 做 literal source-over 替换, 避免修补色被旧视角稀释 关闭眼睛后的效果: 同一重绘会话的可见结果消失; hidden draft 不应单独显现
合并 UV 图层
底层身份: uv, role=merged-uv 数据: 已展平 straight-RGBA PNG、merge version 视觉效果/合成规定: 按 UV0 直接采样, 是当前投影/UV 源的非破坏快照; 不含实时 PBR 灯光 关闭眼睛后的效果: 下方已被合并且已隐藏的源层不会自动顶替它, 除非用户重新显示
投射贴图 · 底/右/前/后/左/顶
底层身份: projected, 通常无特殊 role 数据: 每层一张生成图 + 独立 capture camera/depth/object matrix 视觉效果/合成规定: 普通层进入几何质量门控和 Top-3 多视图合成; 不同名称只表示捕获方向, 不是六套算法 关闭眼睛后的效果: 该相机候选被移出实时与后续 bake 输入
内容识别修补 1
底层身份: uv, role=content-aware-underlay 数据: 仅缺口 texel 有 alpha 的稀疏 PNG 视觉效果/合成规定: 永远作为 underlay: 只补没有可靠投影的区域, 不覆盖普通投影/局部重绘 关闭眼睛后的效果: 缺口重新透明/露出更下层颜色

空白图层

- 底层身份: uv, 无图或空 imageUrl
- 数据: 目标占位
- 视觉效果/合成规定: 本身无视觉像素, 可被“合并到选中空白 UV 层”写入
- 关闭眼睛后的效果: 无变化

隐藏重绘草稿 (面板不列出)

- 底层身份: uv, role=local-repaint-draft
- 数据: 回贴会话目标/兼容数据
- 视觉效果/合成规定: 只供实现和后续 UV commit 使用, 不能被当成第二份用户可见效果
- 关闭眼睛后的效果: 由配对事务管理

Normal 层

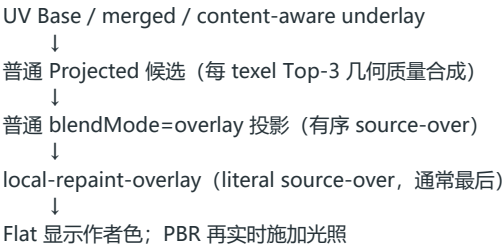
- 底层身份: normal 或模型 normalMap
- 数据: 法线纹理
- 视觉效果/合成规定: 供 Normal/PBR 显示或导出; 不进入 BaseColor Top-3
- 关闭眼睛后的效果: 法线效果消失或回退模型原法线

Patch 层

- 底层身份: patch schema 兼容类型
- 数据: 取决于旧/专用路径
- 视觉效果/合成规定: 当前截图 BaseColor 主栈不把它作为普通 UV/Projected 合成源; 修改前必须先找到创建者和消费者
- 关闭眼睛后的效果: 不得仅凭名字推断

16.3 当前层序合成规则

面板 order=0 在最上方, 但底层合成按语义分组, 不是简单把所有 PNG 从下往上做同一种 alpha blend:



额外不变量:

- 一个可见 merged-uv 是展平边界; 位于它下方、已被该合并消费的投影通常保持隐藏, 避免同一像素重复叠加。
- UV 预览合成目前以 source-over + opacity 为主, 局部重绘 UV overlay 被强制置顶; 不能假定 schema 中的 multiply/screen/soft-light 已在所有 UV 路径生效。
- 普通 Projected 的 strength 改几何质量曲线; opacity 改 coverage。两者不是同一个滑杆。
- maskUrl 的黑白灰是连续 coverage; 不是只按 0/1 读取。

17. 投影算法审计（关键模块）

17.1 调用图

- Generation + Capture
- layerStore.addProjectedLayerFromGeneration
 - SceneRoot 收集当前对象可见 Layer 并判定 compositeRole
 - create/updateProjectedLayerStackMaterial
 - ProjectedLayerMaterial 加载 image/mask/depth/normal 与捕获矩阵
 - ALG-PROJ-001 坐标重投影
 - ALG-PROJ-002/003 coverage 与表面可见性
 - ALG-PROJ-004 Top-3 或 ALG-PROJ-005 Overlay 合成
 - 当前视口 Flat/PBR 输出

17.2 算法登记卡

ALG-PROJ-001 捕获锁定重投影

- 调用者:** 实时材质、UV bake、画笔回贴
- 输入 → 输出:** current world point + capture camera/object matrix → capture image UV/depth
- 核心规则:** clip=P_capture·V_capture·(M_capture·inverse(M_current))·world_current; 先恢复生成时对象姿态再投影
- 失败/回退:** 矩阵/相机缺失则该层不能作为可靠投影, 不能偷偷改用当前相机

ALG-PROJ-002 连续覆盖门控

- 调用者:** projection shader
- 输入 → 输出:** source/mask/angle/edge/opacity → coverage
- 核心规则:** opacity × sourceAlpha × mask × angle × visibility × facing × edgeFade; 所有灰度保留 feather
- 失败/回退:** coverage 低于阈值视为空, 不用二值膨胀掩盖根因

ALG-PROJ-003 深度-法线表面可见性

- 调用者:** projection shader
- 输入 → 输出:** projected depth + capture depth/normal → visibility/quality
- 核心规则:** 3×3 深度/法线支持; 线性 view depth; 掠射角放宽但有限; surface lock 以深度和最小 face-on 为主
- 失败/回退:** 缺 depth 进入角度退化路径; 不得伪造 depth=可见

ALG-PROJ-004 Top-3 一致性合成

- 调用者:** 普通投影预览/UV 质量合成
- 输入 → 输出:** N 个普通候选 → 1 个颜色 + confidence
- 核心规则:** 每 texel 保留分数最高 3 个; 颜色离群降权; 强质量权重与 coverage 残差混合; 明显胜者收敛
- 失败/回退:** GPU/WebGPU parity 不合格时使用 CPU exact 组合, 不降低目标分辨率

ALG-PROJ-005 有序覆盖合成

- 调用者:** overlay/local repaint
- 输入 → 输出:** ordered overlays → color
- 核心规则:** 普通 overlay alpha=coverage*(0.75+0.25*qualityFade); local repaint alpha=coverage, 逐层 source-over
- 失败/回退:** source/mask 未就绪时不发布半成品层

ALG-PROJ-006 纹理驻留预算

调用者: ProjectedLayerMaterial

输入 → 输出: 多层大图 → GPU textures

核心规则: 小栈 direct samplers, 大栈 WebGL2 arrays; 颜色/Mask/Depth/Normal 各有预算并分条上传

失败/回退: 超预算按预览上限缩放, 生产 UV bake 仍保留调用方分辨率

17.3 生产参数基线

参数	实时/GPU 当前值	含义
后向硬拒绝	$N \cdot V < -0.35$	明确背面不采样
无 depth 角度覆盖	$\text{smoothstep}(-0.62, -0.18, N \cdot V)$	没有遮挡证据时保守羽化
有 depth 绝对面向	$0.02 \rightarrow 0.38$	depth-backed 候选的角度权重
quality 角度段	$0.02 \rightarrow 0.25$	质量而非是否覆盖
强度 gamma	$4 / \text{clamp}(\text{strength}, 0.25, 3)$	strength 越大, 角度质量惩罚越弱
coverage feather	threshold 0.02, 到 0.12, floor 0.08	连续边缘而非硬切
NDC depth epsilon	0.0025	裁剪深度数值保护
线性 view depth 容差	$\text{max}(0.00625, \text{viewDepth} * 0.00075)$	随距离增长
深度邻域	中心 + 8 邻域	抗空洞/采样偏差
掠射放宽上限	$5 \times$	防止薄面断裂但不无限穿透
捕获法线一致	$0.72 \rightarrow 0.92$	低一致候选降为不可见
普通 facing	minimum 0.01, full 0.20	面朝捕获相机程度
surface lock facing	$0.015 \rightarrow 0.06$, safe 0.25	局部替换防止翻到相邻/背面
图像边缘	coverage 0.015, quality 0.035	源图边界羽化
local repaint 最小 face-on	0.08	交互回贴额外地平线保护

17.4 Top-3 权重精确定义

- 候选入选分数: $\text{score} = \text{max}(\text{quality}, \text{coverage} * 0.08)$ 。
- 置信度: $\text{confidence} = 1 - \prod (1 - \text{coverage}_i)$ 。
- 先以质量权重求临时基色; 每个候选再乘颜色一致性 $0.35 + 0.65 * \exp(-\text{distance}^2 / 0.22^2)$ 。
- 强权重使用 $\text{quality}^{2.4}$, 软权重使用 coverage; 最终约 $0.8 * \text{strong} + 0.2 * \text{soft}$ 。
- 第一/第二质量比从 1.45→2.6、质量差从 0.05→0.2 共同决定向第一名收敛程度。
- 局部重绘不进入这套平均; 否则旧投影会把用户刚画的新颜色重新混回去。

17.5 GPU 资源预算

资源	预算	常用预览最大边
Color	64 MB	1536
Mask	128 MB	2048
Depth	224 MB	2048
Normal	64 MB	1024

预算只影响实时驻留/预览策略。任何改变预算、采样器布局、Top-3 常量或 visibility gate 的修改都属于 L2，必须同时验证实时预览、UV bake、局部重绘和内容修补。

18. UV 合成与合并算法审计（关键模块）

18.1 右侧剪刀的完整事务

LayersPanelActions 剪刀

- `EditorPage.mergeLayersToUvLayer(sourceIds, optionalBlankUvid)`
- 校验活动对象、模型、UV0、源层和分辨率
- 分类普通 projected / local repaint / flattenable UV / content-aware underlay
- 把 repaint 的可编辑 mask 烘进 source alpha
- `bakeVisibleProjectedLayersToTexture`
 - GPU UV0 triangle raster（每个投影层）
 - Top-3 quality compositor
 - ordered local/overlay compositor
 - topology/seam/gutter postprocess
- 把已有 UV 与 content-aware 作为 underlay 合到投影结果下面
- `encodeRgbaPngObjectUrl` (straight RGBA)
- workspace 保存 layer PNG 并预热解码/GPU
- `layerStore.mergeLayersIntoUvLayer`
- 隐藏源层，插入/写入 visible merged-uv，立即保存

18.2 算法登记卡

ALG-UV-001 合并输入冻结与分类

输入 → 输出: selected Layer IDs → immutable merge sources

当前规则: 只处理当前对象；校验 UV0；普通投影、literal repaint、普通 UV、content-aware 分组；resolution 取当前 1K/2K/4K/8K

主要代码: `EditorPage.mergeLayersToUvLayer`

ALG-UV-002 UV0 三角投影栅格

输入 → 输出: mesh UV0 + projection → per-layer RGBA/coverage/quality

当前规则: GPU 默认把几何三角形画入 UV RT，重建 world position/normal 后调用投影门控

主要代码: `gpuUvBakeRenderer.ts`

ALG-UV-003 Runtime 可见性补获

输入 → 输出: stale/missing capture depth → refreshed depth

当前规则: 对象矩阵或 depth 不可靠时按原 capture camera 重捕 depth，默认不强制 normal，最大边 2048

主要代码: `bakeProjectedLayerToTexture.ts`

ALG-UV-004 Top-3 质量合成

输入 → 输出: normal projected rasters → authored albedo RGBA

当前规则: 与第 17 节权重语义对齐；Worker 负责 exact quality blend；WebGPU 只有通过 parity 才接受

主要代码: `qualityBlend.worker.ts`、`WebGPU compositor`

ALG-UV-005 Literal Overlay 批处理

输入 → 输出: final contiguous repaint overlays → RGBA

当前规则: local repaint 使用 coverage-alpha 且在普通层后有序叠加；批处理失败退为逐层

主要代码: `projectedOverlayComposition.ts`

ALG-UV-006 拓扑约束后处理

输入 → 输出: sparse UV result + mesh topology → seam-safe RGBA

当前规则: 按调用参数执行 gutter、interior gap、enclosed hole、physical seam reconcile、锐化；不得串到不相关 UV island

主要代码: `dilation.ts`、`uvSeamReconciliation.ts`

ALG-UV-007 Under 合成

输入 → 输出: UV/content repair + projected result → final merge pixels

当前规则: 投影在上; 普通 UV 和内容修补在下。WebGPU under compositor 失败时 CPU compositeRgbaUnderInPlace

主要代码: webGpuRgbaComposite.ts、mergeUvComposition.ts

ALG-UV-008 Straight-RGBA 发布

输入 → 输出: raw RGBA → PNG URL/asset + merged Layer

当前规则: 不预乘 RGB; 即使 alpha=0 也保留 RGB, 避免边缘黑线; workspace 先保存/预热再原子发布

主要代码: PNG encoder、mergeLayersIntoUvLayer

18.3 GPU 与 CPU 回退不是同一参数副本

项目	GPU 生产路径	CPU uvRasterizer 参考/回退
栅格方式	GPU triangle raster	每 texel 4 个 subpixel sample + barycentric interpolation
后向硬拒绝	-0.35	-0.35
角度覆盖段	依 depth 路径, 主基线见第 17 节	约 -0.25 → 0.08
quality 角度段	0.02 → 0.25	0.02 → 0.25
depth epsilon	NDC 0.0025 + linear tolerance	CPU 路径约 0.08 的自身容差语义
coverage threshold	约 0.02	约 0.025; source reject 约 0.01
定位	当前默认/生产	诊断与失败回退

禁止在普通修复中把两列“顺手统一”。要统一必须建立 CPU/GPU golden 对比, 解释每个差异是数值表示差异还是产品语义差异。

18.4 UV 输出边界

- 当前只使用单套 uv attribute (UV0) , 不支持 UDIM。
- 输出是 BaseColor straight RGBA; texture.flipY=false 是当前 Three/glTF 约定。
- 合并不保存视口 PBR 环境、曝光、主光或线框效果。
- 内容修补作为稀疏 underlay, 不能覆盖已有可靠投影。
- selected source layers 在成功发布后隐藏但不删除, 便于撤销、检查血缘和重新合并。
- GPU 失败可以切 CPU, 但不得把 4K 静默降成 1K 后仍宣称成功完成 4K 合并。

19. 局部重绘算法审计（关键模块）

19.1 主路径状态机

- Idle
- SelectMask（表面选择，可选；空选择=全图生成范围）
 - FreezeCapture（冻结方形相机 + object matrix）
 - GenerateRunning（白模/材质参考/Flat 视口三输入）
 - ResultReady（raw + depth + archived mask + viewport ref）
 - Harmonize（仅蒙版内边界 RGB）
 - ApplyBrush（用户在表面决定真正回贴范围）
 - PersistProjectedOverlay（当前生产路径）
 - Merge/Export 时才 UV bake

19.2 算法登记卡

ALG-LR-001 冻结视角选择重投影

输入 → 输出：多相机表面笔画 + current camera → 2048 square mask + frozen camera

当前精确规则：点击生成边界固定 aspect=1 相机和对象矩阵；已有笔画按该相机重新投影；没有用户 mask 时创建全白 2048 mask

主要入口：GeneratePanel.handleLocalRepaintGenerate、Viewport paint mask capture

ALG-LR-002 三输入 ModelView 生成

输入 → 输出：white model + material reference + flat viewport → raw result

当前精确规则：输入顺序固定：1 white-model.png, 2 多视图材质参考, 3 viewport-reference.png; provider=modelview-int8, workflow=2026.08.17-a9dbbca-flux2-klein-truev3-3input-r2

主要入口：GeneratePanel.tsx、modelview client

ALG-LR-003 并行深度保护

输入 → 输出：frozen camera/object → 1K linear-view depth

当前精确规则：与远端生成并行；depth 失败允许保留生成结果但必须 warning，回贴几何保护降级

主要入口：capture depth

ALG-LR-004 边界谐调 v2

输入 → 输出：raw result + flat viewport ref + archived mask → harmonized PNG

当前精确规则：只修改 mask 内部边界 RGB, alpha 始终来自 raw；失败/旧工程回 raw；结果连同 version=2 和宽度元数据保存

主要入口：getLocalRepaintProjectionImage、seamHarmonizationCore.ts

ALG-LR-005 表面画笔重投影

输入 → 输出：pointer hit + tangent brush + archived mask → live projection mask

当前精确规则：raycast 命中模型表面，将圆形笔刷投向 source image UV；限制地平线拉伸，abs(face-view)>=0.08；archived generation mask 是允许区

主要入口：ViewportCanvas.tsx

ALG-LR-006 低延迟实时覆盖

输入 → 输出：source + live mask + depth/camera → viewport overlay

当前精确规则：ignoreSourceAlpha=true, coverage 只由用户 mask/几何控制；surface-locked-v1；使用独立 live GPU overlay 避免每笔重烘焙

主要入口：ViewportCanvas.tsx、Projected material

ALG-LR-007 延迟投影持久化

输入 → 输出: completed brush mask → visible projected repaint + hidden draft

当前精确规则: 当前开关关闭 interactive UV bake; latest-wins idle commit, visible layer 标记 needsRebake=true, 保存 source/mask/depth/camera/replacement target/seam 元数据

主要入口: ViewportCanvas.tsx

ALG-LR-008 禁用的即时 UV 分支

输入 → 输出: brush stroke → UV overlay

当前精确规则: 代码仍可按最多 4K、coverage-alpha、透明/拓扑约束烘焙, 但 LOCAL_REPAINT_INTERACTIVE_UV_BAKE_ENABLED=false

主要入口: ViewportCanvas.tsx

ALG-LR-009 Provider 兼容编辑

输入 → 输出: image + edit/protect/hole masks + references → edited layer/repair UV

当前精确规则: LocalRepaintDialog 的旧/兼容路径; 与 ModelView 三输入路径独立

主要入口: LocalRepaintDialog.tsx、image edit provider

19.3 远端没有收到什么

主路径本地选择 mask 不作为第 4 张图发送给远端 ModelView。它的作用是:

- 1. 归档用户期望区域;
- 2. 回贴时限制可画区域;
- 3. 与 capture depth 一起保护模型表面;
- 4. 合并/导出时决定 literal overlay coverage.

因此“远端生成图似乎改了更多区域”不等于最终模型会全部采用; 最终采用范围由第 3 步应用画笔和本地 mask 决定。若未来把 mask 发给远端, 属于远端输入契约 Major 变化, 必须升级 workflow ID 或兼容版本。

19.4 Seam Harmonization v2 参数

项目	当前规则
hard mask	$\max(R,G,B)/255 \times \alpha \geq 0.5$
默认请求宽度	短边 1.8%, clamp 8-32px
几何上限	不超过最大内部距离的 1/3, 至少 2px; 空、全满或过薄 mask 跳过
距离	近似 8 邻域 distance transform
medium radius	$\max(2, \text{width}/6)$
low radius	$\max(\text{medium}+1, \text{width}/2)$
颜色处理	边界内由参考低/中/细频带平滑过渡到生成图; 深内部和 mask 外不改
Lab 全局校色	默认关闭, 除非显式 option 开启
alpha	永远保留 raw generated alpha

19.5 截图中“一行重绘”的双层实现

用户看到	内部层	原因
一行 局部重绘 · 局部替换	visible projected/local-repaint-overlay	保存当前非破坏回贴效果，可继续按 capture 表面显示
看不到的配对层	hidden uv/local-repaint-draft	作为会话目标和兼容 UV commit 位置

图层眼睛、删除、撤销和工程保存必须把这两者作为一个重绘事务管理。只改可见层而遗留 draft，会造成刷新恢复、合并或导出时出现幽灵数据。

20. Ctrl+S 保存实现审计

20.1 一句话答案

是 JSON 工程清单加独立二进制资产：Ctrl/Cmd+S 最终通过 HTTP PUT /api/projects/:projectId 保存 project.liclick.json；模型、参考图、Capture、Generation、Layer PNG、Bake 和缩略图先写到工程的 assets/*，JSON 只保存归一化后的相对路径与算法元数据。

20.2 精确调用链

- shortcutStore: project.save = Primary + KeyS
- EditorPage window keydown (对话框打开或 editorTaskRunning 时拦截)
 - handleManualSave(showSuccessToast=true)
 - getProjectSnapshot(refreshThumbnail=true)
 - prepareProjectForWorkspaceSave(structuredClone)
 - mapWithConcurrency(..., 3) 持久化所有独立资产
 - frontend workspaceSaveQueueRef 串行化
 - workspaceApiClient.saveProject
 - PUT /api/projects/:projectId, 30s timeout, JSON body
 - server handleProjectsRoute + requireAuth
 - projectFileService.saveProject, 以 userId:projectId 串行化
 - 冲突/误清空保护 + 相对路径清洗
 - 原子替换 project.liclick.json
 - 写 autosave, 最多保留 5 份

20.3 快照包含什么

类别	快照来源/内容	磁盘形式
Project 元数据	id/name/time/settings/currentMode/workspa ceVersion/active IDs/pipeline 等	project.liclick.json JSON 字段
Objects	sceneStore.objects, sourcePath、transfor m、visibility 等	JSON；模型文件在 assets/models
Layers	layerStore.layers, 完整第 16 节字段	JSON；image/mask/depth/rendered mask 在 assets/layers
Generations	当前 project 的 generationStore.generations	JSON；result image 在 assets/generations
Captures	Project captures 的 color/mask/depth/normal	JSON + assets/captures PNG
References	referenceStore.references	JSON + assets/references
Baked textures	Project bakedTextures	JSON + assets/baked
Thumbnail	显式 Ctrl+S 刷新标准工程缩略图；后台立即保存不做同步 WebGL 编码	JSON path + capture asset
编辑器 Undo	不写入工程；当前 history persistence 是 no-op	仅当前页面 session

20.4 工程目录和文件格式

```
<workspaceDir>/users/<userId>/projects/<slug>/  
project.liclick.json  
assets/  
models/  
references/  
captures/  
generations/  
layers/  
baked/  
exports/  
thumbnails/  
autosave/  
autosave-<timestamp>.liclick.json # 最多 5 份
```

Layer 图像通常为 PNG；模型保留其持久化资产格式；工程主文件是 UTF-8 JSON。服务端写入 `<destination>.<uuid>.tmp` 后 rename，不先删除旧文件；Windows 临时占用会按 25/50/100/200/400/800ms 重试。

20.5 保存保护与用户可见语义

- 前端同一页面保存排队，避免两个快照同时写；若保存期间状态继续变化，旧结果不会错误标成 Saved，并可再保存最新快照。
- 服务端拒绝较旧 `updatedAt` 覆盖更新工程，返回 409；前端会尝试重新抓最新快照。
- 已有对象/图层的工程不能被一个意外空快照清空，除非 `deletedObjectIds` 明确表示删除意图。
- 保存成功只表示工程和资产已持久化，不表示源码算法版本已提交 Git。
- 页面上的 Saved 对应工作区持久化状态；算法历史仍以 Git、CHANGELOG.md、docs/changes 和 ADR 为准。

21. 算法调用审计的维护格式

以后新增或修改任何会改变像素、图层、保存或导出的功能，必须在本文档登记一张算法卡，至少包含：

必填项	要求
ALG ID + 中文名	ID 永久稳定；实现文件移动时不重命名
UI/外部调用者	指明按钮、快捷键、自动触发器或 API
主入口	函数/服务/Worker/Shader 符号
输入契约	坐标空间、颜色空间、分辨率、alpha、相机、对象矩阵、版本
步骤与公式	生产默认参数；不能只写“优化融合”
输出契约	Layer type/role、文件格式、straight/premultiplied alpha、持久化字段
分支	GPU/CPU/Worker/远端/禁用分支分别写清
失败回退	是否降级、是否继续、是否可观测；禁止静默改变语义
缓存/失效	cache key 包含什么；哪些 Layer 改动触发 rebake
测试/证据	单测、golden、视觉样例、性能或重开工程验证
版本影响	Patch/Minor/Major，是否迁移旧工程

审计问题的固定回答顺序是：UI ID → 用例 → ALG ID → 输入 → 核心参数 → Layer 输出 → 磁盘输出 → 回退 → 测试。例如问“剪刀做了什么”，不能只回答“合并图层”，必须引用 UI-09 → ALG-UV-001~008 → merged-uv PNG → 隐藏源层 → Ctrl+S 保存。

22. 修订历史

1.0.0
日期: 2026-08-26 变更: 建立系统模块、当前算法基线、版本和强制变更流程 变更记录: CHG-20260826-GOVERNANCE-BASELINE
1.1.0
日期: 2026-08-26 变更: 按编辑器全界面建立 UI 模块索引, 补齐图层语义、投影、UV 合成、局部重绘和 Ctrl+S 的代码级调用审计 变更记录: CHG-20260826-GOVERNANCE-BASELINE
1.2.0
日期: 2026-08-26 变更: 为 23 个核心算法补齐中英文基础名称、稳定 ID、规范版本、真实运行标记和兼容性升级规则; 建立 DOCX/PDF 正式维护手册基线 变更记录: CHG-20260826-GOVERNANCE-BASELINE